

C++ 八股文

一、面向对象核心

1. 封装、继承、多态的区别与联系？

封装：将数据和方法包装在类中，通过访问控制（public/protected/private）隐藏内部实现细节，只暴露公共接口。

继承：子类继承父类的属性和方法，实现代码复用，建立类之间的层次关系。

多态：同一接口，不同实现。分为编译时多态（函数重载、模板）和运行时多态（虚函数）。

```
#include <iostream>
using namespace std;

// 封装: 通过 private 隐藏数据, public 提供接口
class Animal {
private:
    string name; // 私有成员, 外部不能直接访问
protected:
    int age;      // 保护成员, 子类可以访问
public:
    Animal(string n, int a) : name(n), age(a) {}
    string getName() const { return name; } // 公共接口
    virtual void speak() { cout << name << " makes a sound" << endl; }
    virtual ~Animal() {} // 虚析构函数, 防止内存泄漏
};

// 继承: Dog 继承 Animal
class Dog : public Animal {
public:
    Dog(string n, int a) : Animal(n, a) {}
    // 多态: 重写父类虚函数
    void speak() override { cout << getName() << " barks" << endl; }
};

class Cat : public Animal {
public:
    Cat(string n, int a) : Animal(n, a) {}
    void speak() override { cout << getName() << " meows" << endl; }
};

// 多态的使用: 基类指针指向派生类对象
void makeAnimalSpeak(Animal *animal) {
    animal->speak(); // 运行时决定调用哪个版本
}

int main() {
    Dog dog("Buddy", 3);
    Cat cat("Kitty", 2);

    makeAnimalSpeak(&dog); // Buddy barks
    makeAnimalSpeak(&cat); // Kitty meows
}
```

```
    return 0;
}
```

```
/*
运行结果：
Buddy barks
Kitty meows
*/
```

延伸：三种继承方式的区别：

继承方式	public 成员变为	protected 成员变为	private 成员
public 继承	public	protected	不可访问
protected 继承	protected	protected	不可访问
private 继承	private	private	不可访问

2. 虚函数和虚函数表的原理？

虚函数：用 `virtual` 关键字修饰的成员函数，允许派生类重写，实现运行时多态。

虚函数表 (vtable)：编译器为每个包含虚函数的类生成一张表，存储虚函数的地址。每个对象有一个隐藏的虚表指针 (vptr)，指向该类的虚函数表。

```
#include <iostream>
using namespace std;

class Base {
public:
    virtual void func1() { cout << "Base::func1" << endl; }
    virtual void func2() { cout << "Base::func2" << endl; }
    void func3() { cout << "Base::func3" << endl; } // 非虚函数
};

class Derived : public Base {
public:
    void func1() override { cout << "Derived::func1" << endl; }
    // func2 未重写，继承 Base 的版本
};

int main() {
    Base b;
    Derived d;

    Base *p = &d;
    p->func1(); // Derived::func1 (通过 vtable 调用)
    p->func2(); // Base::func2 (继承的, vtable 中指向 Base::func2)
    p->func3(); // Base::func3 (非虚函数, 编译时确定)

    cout << "sizeof(Base) = " << sizeof(Base) << endl; // 8 (vptr)
```

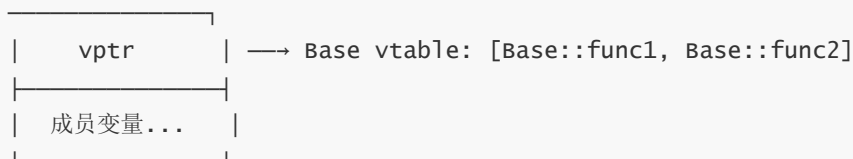
```
cout << "sizeof(Derived) = " << sizeof(Derived) << endl; // 8 (共享 vptr)

return 0;
}
```

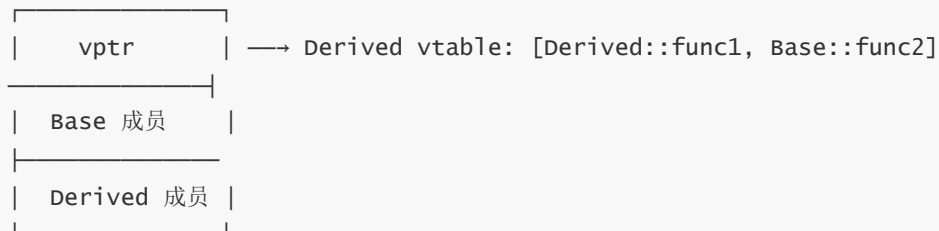
```
/*
运行结果:
Derived::func1
Base::func2
Base::func3
sizeof(Base) = 8
sizeof(Derived) = 8
*/
```

延伸：虚函数表的内存布局：

Base 对象的内存布局：



Derived 对象的内存布局：



关键问题：

问题	答案
构造函数可以是虚函数吗？	不可以，构造时 vptr 还未初始化
析构函数可以是虚函数吗？	可以，且基类析构函数应该设为虚函数
静态函数可以是虚函数吗？	不可以，静态函数不属于对象
内联函数可以是虚函数吗？	可以声明，但通过指针调用时不能内联

3. 为什么基类析构函数要设为虚函数？

当通过基类指针删除派生类对象时，如果基类析构函数不是虚函数，只会调用基类的析构函数，导致派生类部分未被销毁，造成内存泄漏。

```
#include <iostream>
using namespace std;

class Base {
```

```

public:
    Base() { cout << "Base 构造" << endl; }
    ~Base() { cout << "Base 析构" << endl; } // 非虚析构
};

class Derived : public Base {
    int *data;
public:
    Derived() : data(new int[100]) { cout << "Derived 构造" << endl; }
    ~Derived() {
        delete[] data;
        cout << "Derived 析构" << endl;
    }
};

int main() {
    Base *p = new Derived();
    delete p; // 只调用 Base::~~Base(), Derived 的 data 泄漏!

    return 0;
}

```

```

/*
运行结果（危险）：
Base 构造
Derived 构造
Base 析构
（Derived 析构未被调用，data 内存泄漏）
*/

```

正确做法：

```

class Base {
public:
    virtual ~Base() { cout << "Base 析构" << endl; } // 虚析构
};

```

```

/*
正确运行结果：
Base 构造
Derived 构造
Derived 析构
Base 析构
*/

```

延伸：纯虚析构函数：

```

class Abstract {
public:
    virtual ~Abstract() = 0; // 纯虚析构
};

// 纯虚析构必须有定义
Abstract::~~Abstract() {
    cout << "Abstract 析构" << endl;
}

```

4. 深拷贝和浅拷贝的区别？

浅拷贝：默认拷贝构造函数和赋值运算符，只复制指针的值，两个对象共享同一块内存。

深拷贝：手动实现拷贝构造和赋值运算符，为每个对象分配独立内存。

```

#include <iostream>
#include <cstring>
using namespace std;

class String {
    char *data;
    int len;
public:
    String(const char *str) {
        len = strlen(str);
        data = new char[len + 1];
        strcpy(data, str);
    }

    // 浅拷贝（危险）：
    // String(const String &other) : data(other.data), len(other.len) {}

    // 深拷贝（正确）：
    String(const String &other) {
        len = other.len;
        data = new char[len + 1];
        strcpy(data, other.data);
    }

    // 深拷贝赋值运算符
    String& operator=(const String &other) {
        if (this != &other) { // 防止自赋值
            delete[] data;
            len = other.len;
            data = new char[len + 1];
            strcpy(data, other.data);
        }
        return *this;
    }

    ~String() { delete[] data; }

    void print() const { cout << data << endl; }
}

```

```
};

int main() {
    String s1("Hello");
    String s2 = s1; // 调用深拷贝构造函数

    s1.print(); // Hello
    s2.print(); // Hello

    // 修改 s1 不影响 s2 (深拷贝)
    return 0;
}
```

延伸: C++11 移动语义 (避免深拷贝的开销) :

```
class String {
    char *data;
    int len;
public:
    // 移动构造函数
    String(String &&other) noexcept : data(other.data), len(other.len) {
        other.data = nullptr; // 转移所有权
        other.len = 0;
    }

    // 移动赋值运算符
    String& operator=(String &&other) noexcept {
        if (this != &other) {
            delete[] data;
            data = other.data;
            len = other.len;
            other.data = nullptr;
            other.len = 0;
        }
        return *this;
    }
};
```

5. 构造函数和析构函数的执行顺序?

构造顺序: 基类 → 成员对象 → 派生类

析构顺序: 派生类 → 成员对象 → 基类 (与构造相反)

```
#include <iostream>
using namespace std;

class Member {
    string name;
public:
    Member(string n) : name(n) { cout << "Member " << name << " 构造" << endl; }
    ~Member() { cout << "Member " << name << " 析构" << endl; }
};
```

```

class Base {
    Member m_base;
public:
    Base() : m_base("Base") { cout << "Base 构造" << endl; }
    virtual ~Base() { cout << "Base 析构" << endl; }
};

class Derived : public Base {
    Member m_derived;
public:
    Derived() : m_derived("Derived") { cout << "Derived 构造" << endl; }
    ~Derived() override { cout << "Derived 析构" << endl; }
};

int main() {
    Derived d;
    return 0;
}

```

```

/*
运行结果：
Member Base 构造
Base 构造
Member Derived 构造
Derived 构造
Derived 析构
Member Derived 析构
Base 析构
Member Base 析构
*/

```

延伸：多继承的构造顺序：

```

class A { public: A() { cout << "A" << endl; } };
class B { public: B() { cout << "B" << endl; } };
class C : public A, public B {
    Member m;
public:
    C() : m("C") { cout << "C" << endl; }
};
// 顺序: A → B → Member → C

```

二、C++11 新特性

6. 智能指针有哪些？各自的区别？

C++11 引入三种智能指针，自动管理内存，避免手动 delete：

智能指针	特点	使用场景
<code>unique_ptr</code>	独占所有权，不可复制	单一所有者场景

智能指针	特点	使用场景
<code>shared_ptr</code>	共享所有权，引用计数	多个所有者场景
<code>weak_ptr</code>	不增加引用计数，观察 <code>shared_ptr</code>	解决循环引用

```

#include <iostream>
#include <memory>
using namespace std;

class MyClass {
public:
    MyClass() { cout << "构造" << endl; }
    ~MyClass() { cout << "析构" << endl; }
    void hello() { cout << "Hello" << endl; }
};

// unique_ptr: 独占所有权
void testUnique() {
    unique_ptr<MyClass> p1 = make_unique<MyClass>();
    p1->hello();
    // unique_ptr<MyClass> p2 = p1; // 错误! 不能复制
    unique_ptr<MyClass> p2 = move(p1); // 可以移动
    // p1 现在为空
}

// shared_ptr: 共享所有权
void testShared() {
    shared_ptr<MyClass> p1 = make_shared<MyClass>();
    {
        shared_ptr<MyClass> p2 = p1; // 引用计数 +1
        cout << "use_count: " << p1.use_count() << endl; // 2
    } // p2 销毁, 引用计数 -1
    cout << "use_count: " << p1.use_count() << endl; // 1
} // p1 销毁, 引用计数为 0, 自动 delete

// weak_ptr: 解决循环引用
struct Node {
    shared_ptr<Node> next;
    weak_ptr<Node> prev; // 用 weak_ptr 打破循环
};

void testweak() {
    auto n1 = make_shared<Node>();
    auto n2 = make_shared<Node>();
    n1->next = n2;
    n2->prev = n1; // weak_ptr 不增加引用计数
}

int main() {
    cout << "=== unique_ptr ===" << endl;
    testUnique();

    cout << "=== shared_ptr ===" << endl;
    testShared();
}

```



```

    cout << "=== weak_ptr ===" << endl;
    testweak();

    return 0;
}

```

```

/*
运行结果：
=== unique_ptr ===
构造
Hello
析构
=== shared_ptr ===
构造
use_count: 2
use_count: 1
析构
=== weak_ptr ===
*/

```

延伸：循环引用问题：

```

// 危险：两个 shared_ptr 互相引用，引用计数永远不为 0
struct BadNode {
    shared_ptr<BadNode> next;
    shared_ptr<BadNode> prev; // 应该用 weak_ptr
};

auto n1 = make_shared<BadNode>();
auto n2 = make_shared<BadNode>();
n1->next = n2; // n2 引用计数 = 2
n2->prev = n1; // n1 引用计数 = 2
// 离开作用域后，n1 和 n2 的引用计数都为 1，内存泄漏！

```

7. lambda 表达式的语法和使用场景？

语法： [捕获列表] (参数列表) mutable -> 返回类型 { 函数体 }

捕获方式：

- `[]`：不捕获任何变量
- `[=]`：按值捕获所有变量
- `[&]`：按引用捕获所有变量
- `[x, &y]`：x 按值, y 按引用
- `[this]`：捕获当前对象的 this 指针

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

```

```

int main() {
    int factor = 10;

    // 基本 lambda
    auto add = [](int a, int b) { return a + b; };
    cout << add(3, 5) << endl; // 8

    // 按值捕获
    auto multiply = [factor](int x) { return x * factor; };
    cout << multiply(5) << endl; // 50

    // 按引用捕获（可以修改外部变量）
    auto increment = [&factor]() { factor++; };
    increment();
    cout << factor << endl; // 11

    // 在 STL 算法中使用
    vector<int> nums = {5, 2, 8, 1, 9, 3};

    // 排序
    sort(nums.begin(), nums.end(), [](int a, int b) {
        return a > b; // 降序
    });

    // 遍历
    for_each(nums.begin(), nums.end(), [](int n) {
        cout << n << " ";
    });
    cout << endl;

    // 结合 find_if
    auto it = find_if(nums.begin(), nums.end(), [](int n) {
        return n > 7;
    });
    if (it != nums.end())
        cout << "找到大于7的数: " << *it << endl;

    return 0;
}

```

```

/*
运行结果:
8
50
11
9 8 5 3 2 1
找到大于7的数: 9
*/

```

延伸：lambda 与函数指针、std::function 的关系：

```
#include <functional>

// lambda 可以转换为函数指针（无捕获时）
void (*funcPtr)(int) = [](int x) { cout << x << endl; };

// std::function 可以存储任何可调用对象
function<int(int, int)> f1 = [](int a, int b) { return a + b; };
function<int(int, int)> f2 = add; // 普通函数
function<int(int, int)> f3 = [](int a, int b) { return a * b; };
```

8. C++11 还有哪些重要新特性?

```
#include <iostream>
#include <vector>
#include <unordered_map>
#include <tuple>
#include <thread>
using namespace std;

// 1. auto 类型推导
void testAuto() {
    auto x = 42;           // int
    auto y = 3.14;         // double
    auto v = vector<int>{1, 2, 3};
    for (auto &item : v) { // 范围 for 循环
        cout << item << " ";
    }
    cout << endl;
}

// 2. nullptr 替代 NULL
void testNullptr() {
    int *p = nullptr; // 类型安全，不会隐式转换为 int
    // int *q = NULL; // NULL 本质是 0，可能引起歧义
}

// 3. 右值引用和移动语义
void testMove() {
    vector<int> v1 = {1, 2, 3, 4, 5};
    vector<int> v2 = move(v1); // 移动语义，避免深拷贝
    // v1 现在为空
    cout << "v2 size: " << v2.size() << endl; // 5
}

// 4. 统一初始化
void testInit() {
    vector<int> v{1, 2, 3, 4, 5};
    unordered_map<string, int> m{"a", 1, "b", 2};
    int x{42}; // 防止窄化转换
    // int y{3.14}; // 编译错误!
}

// 5. constexpr 常量表达式
```

```

constexpr int square(int x) {
    return x * x;
}

void testConstexpr() {
    constexpr int result = square(5); // 编译时计算
    int arr[result]; // 可以用在需要常量表达式的地方
}

// 6. 变参模板
void print() { cout << endl; }
template<typename T, typename... Args>
void print(T first, Args... rest) {
    cout << first << " ";
    print(rest...);
}

void testVariadic() {
    print(1, 2.5, "hello", 'c'); // 1 2.5 hello c
}

// 7. std::function 和 std::bind
int add(int a, int b) { return a + b; }

void testFunction() {
    function<int(int, int)> f = add;
    cout << f(3, 5) << endl; // 8

    auto bound = bind(add, 10, placeholders::_1);
    cout << bound(5) << endl; // 15 (10 + 5)
}

// 8. 线程支持
void testThread() {
    auto task = [](int n) {
        for (int i = 0; i < n; i++) {
            cout << "thread: " << i << endl;
        }
    };

    thread t1(task, 5);
    t1.join(); // 等待线程结束
}

int main() {
    testAuto();
    testNullptr();
    testMove();
    testInit();
    testConstexpr();
    testVariadic();
    testFunction();
    // testThread(); // 需要链接 -pthread

    return 0;
}

```

三、STL 标准库

9. vector 的底层原理和扩容机制？

底层结构：连续内存数组，三个指针：`start`、`finish`、`end_of_storage`。

扩容机制：当容量不足时，分配新内存（通常为原来的 2 倍），将旧数据拷贝/移动到新内存，释放旧内存。

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v;
    cout << "初始: size=" << v.size() << ", capacity=" << v.capacity() << endl;

    for (int i = 0; i < 10; i++) {
        v.push_back(i);
        cout << "push " << i << ": size=" << v.size()
            << ", capacity=" << v.capacity() << endl;
    }

    // 预分配内存，避免多次扩容
    vector<int> v2;
    v2.reserve(100); // 只分配内存，不改变 size
    cout << "reserve后: size=" << v2.size() << ", capacity=" << v2.capacity() <<
        endl;

    // 缩容技巧: swap 空 vector
    vector<int>(v).swap(v);
    cout << "swap后: size=" << v.size() << ", capacity=" << v.capacity() << endl;

    return 0;
}
```

```
/*
运行结果（GCC 实现，扩容策略为 2 倍）：
初始: size=0, capacity=0
push 0: size=1, capacity=1
push 1: size=2, capacity=2
push 2: size=3, capacity=4
push 3: size=4, capacity=4
push 4: size=5, capacity=8
push 5: size=6, capacity=8
push 6: size=7, capacity=8
push 7: size=8, capacity=8
push 8: size=9, capacity=16
push 9: size=10, capacity=16
reserve后: size=0, capacity=100
swap后: size=10, capacity=10
*/
```

延伸：vector 迭代器失效问题：

```
vector<int> v = {1, 2, 3, 4, 5};
auto it = v.begin();

// 插入导致扩容，迭代器失效
v.push_back(6); // it 可能失效

// 删除元素，当前迭代器失效
it = v.erase(it); // erase 返回下一个有效迭代器

// 正确遍历删除方式
for (auto it = v.begin(); it != v.end(); ) {
    if (*it % 2 == 0)
        it = v.erase(it);
    else
        ++it;
}
```

10. map 和 unordered_map 的区别？

对比项	map	unordered_map
底层结构	红黑树	哈希表
有序性	按键有序	无序
查找时间	O(log n)	O(1) 平均
插入时间	O(log n)	O(1) 平均
内存占用	较少	较多（哈希表需要额外空间）
适用场景	需要有序遍历	快速查找

```
#include <iostream>
#include <map>
#include <unordered_map>
using namespace std;

int main() {
    // map: 有序
    map<string, int> m;
    m["banana"] = 3;
    m["apple"] = 1;
    m["cherry"] = 2;

    cout << "map (有序):" << endl;
    for (auto &p : m)
        cout << p.first << ": " << p.second << endl;
    // 输出: apple: 1, banana: 3, cherry: 2

    // unordered_map: 无序但更快
    unordered_map<string, int> um;
```

```

um["banana"] = 3;
um["apple"] = 1;
um["cherry"] = 2;

cout << "\nunordered_map (无序):" << endl;
for (auto &p : um)
    cout << p.first << ": " << p.second << endl;

// 查找
auto it = m.find("apple");
if (it != m.end())
    cout << "\n找到 apple: " << it->second << endl;

// 统计出现次数
cout << "count: " << m.count("apple") << endl; // 1

return 0;
}

```

延伸：自定义哈希函数：

```

struct PairHash {
    size_t operator()(const pair<int, int> &p) const {
        return hash<int>()(p.first) ^ (hash<int>()(p.second) << 1);
    }
};

unordered_map<pair<int, int>, int, PairHash> memo;
memo[{1, 2}] = 10;

```

11. vector、list、deque 的区别和选择？

对比项	vector	list	deque
内存布局	连续	双向链表	分段连续
随机访问	O(1)	O(n)	O(1)
头尾插入	尾 O(1), 头 O(n)	头尾 O(1)	头尾 O(1)
中间插入删除	O(n)	O(1) (需先找到位置)	O(n)
迭代器失效	扩容时全部失效	只失效被删除的	比较复杂
内存效率	高	低 (每个节点额外指针)	中

```

#include <iostream>
#include <vector>
#include <list>
#include <deque>
using namespace std;

int main() {
    // vector: 适合随机访问、尾部操作

```

```

vector<int> v = {1, 2, 3};
v.push_back(4);           // O(1) 摊销
cout << v[2] << endl;    // O(1) 随机访问

// list: 适合频繁中间插入删除
list<int> l = {1, 2, 3};
auto it = l.begin();
advance(it, 1);           // 移动到第二个元素
l.insert(it, 10);         // O(1) 插入
l.erase(it);              // O(1) 删除

// deque: 适合头尾操作
deque<int> d = {1, 2, 3};
d.push_front(0);          // O(1)
d.push_back(4);           // O(1)
cout << d[2] << endl;    // O(1) 随机访问

return 0;
}

```

选择建议:

- 需要随机访问、尾部操作多 → **vector**
- 需要频繁中间插入删除 → **list**
- 需要头尾都频繁操作 → **deque**

12. STL 迭代器的分类?

输入迭代器 (Input)	→ 只读, 单遍	→ istream_iterator
输出迭代器 (Output)	→ 只写, 单遍	→ ostream_iterator
前向迭代器 (Forward)	→ 读写, 多遍	→ forward_list::iterator
双向迭代器 (Bidirectional)	→ 读写, 前后移动	→ list::iterator
随机访问迭代器 (Random Access)	→ 读写, 任意访问	→ vector::iterator

```

#include <iostream>
#include <vector>
#include <list>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {5, 2, 8, 1, 9};

    // 随机访问迭代器: 支持 +、-、[]、<、>
    auto it = v.begin();
    it += 3;           // 跳到第4个元素
    cout << *it << endl; // 1
    cout << v[3] << endl; // 1 (等价)

    // list 只有双向迭代器: 不支持 +、[]
    list<int> l = {5, 2, 8, 1, 9};
    auto lit = l.begin();
}

```



```

advance(lit, 3); // 只能用 advance
cout << *lit << endl; // 1

// 迭代器适配器
// 反向迭代器
for (auto rit = v.rbegin(); rit != v.rend(); ++rit)
    cout << *rit << " "; // 9 1 8 2 5
cout << endl;

// 插入迭代器
vector<int> src = {1, 2, 3};
vector<int> dst;
copy(src.begin(), src.end(), back_inserter(dst));

return 0;
}

```

13. STL 常用算法有哪些？

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric>
using namespace std;

int main() {
    vector<int> v = {5, 2, 8, 1, 9, 3, 7, 4, 6};

    // 排序
    sort(v.begin(), v.end());
    // v: 1 2 3 4 5 6 7 8 9

    // 查找
    auto it = find(v.begin(), v.end(), 5);
    if (it != v.end())
        cout << "找到 5, 位置: " << distance(v.begin(), it) << endl;

    // 二分查找（需要先排序）
    bool found = binary_search(v.begin(), v.end(), 5);
    cout << "binary_search: " << found << endl;

    // 计数
    int cnt = count(v.begin(), v.end(), 5);
    cout << "count: " << cnt << endl;

    // 求和
    int sum = accumulate(v.begin(), v.end(), 0);
    cout << "sum: " << sum << endl;

    // 最大值/最小值
    cout << "max: " << *max_element(v.begin(), v.end()) << endl;
    cout << "min: " << *min_element(v.begin(), v.end()) << endl;

    // 去重（需要先排序）

```

```

sort(v.begin(), v.end());
auto newEnd = unique(v.begin(), v.end());
v.erase(newEnd, v.end());

// 变换
vector<int> v2 = {1, 2, 3, 4, 5};
transform(v2.begin(), v2.end(), v2.begin(), [](int x) { return x * x; });
// v2: 1 4 9 16 25

// 条件删除
v2.erase(remove_if(v2.begin(), v2.end(), [](int x) { return x > 10; }),
v2.end());
// v2: 1 4 9

return 0;
}

```

四、设计模式

14. 单例模式的实现？

核心思想： 确保一个类只有一个实例，并提供全局访问点。

```

#include <iostream>
#include <mutex>
using namespace std;

// 方法1: 懒汉式（线程安全，C++11 推荐）
class Singleton {
private:
    Singleton() { cout << "Singleton 构造" << endl; }
    Singleton(const Singleton &) = delete;
    Singleton& operator=(const Singleton &) = delete;

public:
    static Singleton& getInstance() {
        static Singleton instance; // C++11 保证线程安全
        return instance;
    }

    void hello() { cout << "Hello from Singleton" << endl; }
};

// 方法2: 饿汉式（线程安全，程序启动时创建）
class EagerSingleton {
private:
    EagerSingleton() {}
    static EagerSingleton instance;
public:
    static EagerSingleton& getInstance() {
        return instance;
    }
};

EagerSingleton EagerSingleton::instance;

```

```
// 方法3: 双重检查锁定 (经典实现)
class DCLSingleton {
private:
    static DCLSingleton *instance;
    static mutex mtx;
    DCLSingleton() {}
public:
    static DCLSingleton* getInstance() {
        if (instance == nullptr) {
            lock_guard<mutex> lock(mtx);
            if (instance == nullptr) {
                instance = new DCLSingleton();
            }
        }
        return instance;
    }
};
DCLSingleton *DCLSingleton::instance = nullptr;
mutex DCLSingleton::mtx;

int main() {
    Singleton &s1 = Singleton::getInstance();
    Singleton &s2 = Singleton::getInstance();
    cout << "s1 == s2: " << (&s1 == &s2) << endl; // 1

    s1.hello();

    return 0;
}
```

```
/*
运行结果:
Singleton 构造
s1 == s2: 1
Hello from Singleton
*/
```

延伸: 单例模式的应用场景:

- 日志管理器
- 配置管理器
- 数据库连接池
- 线程池

15. 工厂模式的实现?

核心思想: 将对象的创建和使用分离, 通过工厂方法创建对象。

```
#include <iostream>
#include <memory>
#include <string>
```

```

#include <unordered_map>
#include <functional>
using namespace std;

// 产品接口
class Button {
public:
    virtual void render() = 0;
    virtual ~Button() = default;
};

// 具体产品
class WindowsButton : public Button {
public:
    void render() override { cout << "渲染 windows 按钮" << endl; }
};

class MacButton : public Button {
public:
    void render() override { cout << "渲染 Mac 按钮" << endl; }
};

class LinuxButton : public Button {
public:
    void render() override { cout << "渲染 Linux 按钮" << endl; }
};

// 简单工厂
class ButtonFactory {
public:
    static unique_ptr<Button> createButton(const string &type) {
        if (type == "windows") return make_unique<WindowsButton>();
        if (type == "mac") return make_unique<MacButton>();
        if (type == "linux") return make_unique<LinuxButton>();
        return nullptr;
    }
};

// 工厂方法模式
class Dialog {
public:
    virtual unique_ptr<Button> createButton() = 0;
    void render() {
        auto btn = createButton();
        btn->render();
    }
};

class WindowsDialog : public Dialog {
public:
    unique_ptr<Button> createButton() override {
        return make_unique<WindowsButton>();
    }
};

class MacDialog : public Dialog {

```

```

public:
    unique_ptr<Button> createButton() override {
        return make_unique<MacButton>();
    }
};

// 注册表工厂（扩展性更好）
class ButtonRegistry {
    unordered_map<string, function<unique_ptr<Button>()>> creators;
public:
    void registerType(const string &type, function<unique_ptr<Button>()> creator)
    {
        creators[type] = creator;
    }

    unique_ptr<Button> create(const string &type) {
        auto it = creators.find(type);
        if (it != creators.end()) return it->second();
        return nullptr;
    }
};

int main() {
    // 简单工厂
    auto btn1 = ButtonFactory::createButton("windows");
    btn1->render();

    // 工厂方法
    unique_ptr<Dialog> dialog = make_unique<MacDialog>();
    dialog->render();

    // 注册表工厂
    ButtonRegistry registry;
    registry.registerType("windows", []() { return make_unique<WindowsButton>(); });
    registry.registerType("linux", []() { return make_unique<LinuxButton>(); });
    auto btn2 = registry.create("linux");
    btn2->render();

    return 0;
}

```

```

/*
运行结果：
渲染 windows 按钮
渲染 Mac 按钮
渲染 Linux 按钮
*/

```

16. 观察者模式的实现？

核心理念：定义对象间的一对多依赖关系，当一个对象状态改变时，所有依赖者自动收到通知。

```
#include <iostream>
#include <vector>
#include <string>
#include <memory>
using namespace std;

// 观察者接口
class Observer {
public:
    virtual void update(const string &message) = 0;
    virtual ~Observer() = default;
};

// 具体观察者
class ConcreteObserver : public Observer {
    string name;
public:
    ConcreteObserver(string n) : name(n) {}
    void update(const string &message) override {
        cout << name << " 收到通知: " << message << endl;
    }
};

// 被观察者（主题）
class Subject {
    vector<shared_ptr<Observer>> observers;
public:
    void addObserver(shared_ptr<Observer> obs) {
        observers.push_back(obs);
    }

    void removeObserver(shared_ptr<Observer> obs) {
        observers.erase(
            remove(observers.begin(), observers.end(), obs),
            observers.end()
        );
    }

    void notify(const string &message) {
        for (auto &obs : observers)
            obs->update(message);
    }
};

// 应用场景：IM 消息推送
class ChatRoom : public Subject {
    string name;
public:
    ChatRoom(string n) : name(n) {}
    void sendMessage(const string &msg) {
        cout << "[" << name << "]" " " << msg << endl;
    }
};
```

```

        notify(msg);
    }
};

int main() {
    ChatRoom room("技术交流群");

    auto user1 = make_shared<ConcreteObserver>("张三");
    auto user2 = make_shared<ConcreteObserver>("李四");
    auto user3 = make_shared<ConcreteObserver>("王五");

    room.addObserver(user1);
    room.addObserver(user2);
    room.addObserver(user3);

    room.sendMessage("大家好! ");

    room.removeObserver(user2);
    room.sendMessage("李四已退出");

    return 0;
}

```

```

/*
运行结果:
[技术交流群] 大家好!
张三 收到通知: 大家好!
李四 收到通知: 大家好!
王五 收到通知: 大家好!
[技术交流群] 李四已退出
张三 收到通知: 李四已退出
王五 收到通知: 李四已退出
*/

```

延伸： Qt 中的信号与槽就是观察者模式的实现：

```

// Qt 信号与槽（观察者模式的封装）
class Button : public QObject {
    Q_OBJECT
signals:
    void clicked();
};

class Label : public QObject {
    Q_OBJECT
public slots:
    void onClicked() { cout << "按钮被点击" << endl; }
};

// 连接：观察者模式
// connect(button, &Button::clicked, label, &Label::onClicked);

```

五、Qt 开发

17. Qt 信号与槽机制的原理？

信号与槽是 Qt 的核心机制，用于对象间通信，是观察者模式的 Qt 实现。

```
#include <QApplication>
#include <QPushButton>
#include <QLabel>
#include <QVBoxLayout>
#include <QWidget>
#include <iostream>
using namespace std;

// 自定义信号和槽
class Counter : public QObject {
    Q_OBJECT
    int value;
public:
    Counter() : value(0) {}

signals:
    void valueChanged(int newValue);

public slots:
    void increment() {
        value++;
        emit valueChanged(value);
    }

    int getValue() const { return value; }
};

class Display : public QObject {
    Q_OBJECT
public slots:
    void onValueChanged(int val) {
        cout << "当前值: " << val << endl;
    }
};

int main(int argc, char *argv[]) {
    QApplication app(argc, argv);

    // 信号与槽连接
    Counter counter;
    Display display;

    // Qt5 语法（类型安全，编译时检查）
    QObject::connect(&counter, &Counter::valueChanged,
                    &display, &Display::onValueChanged);

    counter.increment(); // 输出：当前值：1
    counter.increment(); // 输出：当前值：2
```



```

// 一个信号连接多个槽
QObject::connect(&counter, &Counter::valueChanged,
                [](int val) { cout << "Lambda槽: " << val << endl; });

counter.increment(); // 两个槽都会被调用

return app.exec();
}

```

信号与槽的特点：

- 一个信号可以连接多个槽
- 多个信号可以连接同一个槽
- 信号可以连接信号
- 连接可以是跨线程的（QueuedConnection）
- 类型安全（Qt5 语法）

连接方式：

连接方式	特点
DirectConnection	同线程，直接调用
QueuedConnection	跨线程，事件队列
AutoConnection	默认，自动选择
BlockingQueuedConnection	跨线程，阻塞等待

18. Qt 的内存管理机制？

Qt 采用**对象树**机制管理内存：父对象销毁时自动销毁所有子对象。

```

#include <QObject>
#include <iostream>
using namespace std;

class MyObject : public QObject {
    string name;
public:
    MyObject(string n, QObject *parent = nullptr)
        : QObject(parent), name(n) {
        cout << name << " 构造" << endl;
    }
    ~MyObject() { cout << name << " 析构" << endl; }
};

int main() {
    MyObject *parent = new MyObject("Parent");

    MyObject *child1 = new MyObject("Child1", parent);
    MyObject *child2 = new MyObject("Child2", parent);
}

```

```

    MyObject *grandChild = new MyObject("GrandChild", child1);

    // 删除 parent 时，所有子对象自动删除
    delete parent;

    return 0;
}

```

```

/*
运行结果：
Parent 构造
Child1 构造
Child2 构造
GrandChild 构造
Parent 析构
Child1 析构
GrandChild 析构
Child2 析构
*/

```

注意事项：

- 栈上的 QObject 不需要指定 parent
- 不要同时使用 parent 和智能指针管理同一个对象
- `deleteLater()` 安全删除（等待事件循环处理完再删除）

六、多线程

19. C++ 多线程编程基础？

```

#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <vector>
#include <queue>
using namespace std;

// 1. 基本线程创建
void threadFunc(int id) {
    cout << "线程 " << id << " 运行" << endl;
}

void testBasicThread() {
    vector<thread> threads;
    for (int i = 0; i < 4; i++) {
        threads.emplace_back(threadFunc, i);
    }
    for (auto &t : threads) t.join();
}

// 2. 互斥锁保护共享数据

```

```

mutex mtx;
int sharedCounter = 0;

void increment() {
    for (int i = 0; i < 1000; i++) {
        lock_guard<mutex> lock(mtx); // RAII 锁
        sharedCounter++;
    }
}

void testMutex() {
    thread t1(increment);
    thread t2(increment);
    t1.join(); t2.join();
    cout << "counter: " << sharedCounter << endl; // 2000
}

// 3. 条件变量（生产者-消费者）
queue<int> buffer;
const int MAX_SIZE = 10;
mutex cvMtx;
condition_variable cvNotFull, cvNotEmpty;

void producer(int id) {
    for (int i = 0; i < 5; i++) {
        unique_lock<mutex> lock(cvMtx);
        cvNotFull.wait(lock, []() { return buffer.size() < MAX_SIZE; });
        buffer.push(i);
        cout << "生产者 " << id << " 生产: " << i << endl;
        cvNotEmpty.notify_one();
    }
}

void consumer(int id) {
    for (int i = 0; i < 5; i++) {
        unique_lock<mutex> lock(cvMtx);
        cvNotEmpty.wait(lock, []() { return !buffer.empty(); });
        int val = buffer.front();
        buffer.pop();
        cout << "消费者 " << id << " 消费: " << val << endl;
        cvNotFull.notify_one();
    }
}

void testProducerConsumer() {
    thread prod(producer, 1);
    thread cons(consumer, 1);
    prod.join(); cons.join();
}

int main() {
    cout << "=== 基本线程 ===" << endl;
    testBasicThread();

    cout << "\n=== 互斥锁 ===" << endl;
    testMutex();
}

```

```

    cout << "\n=== 生产者消费者 ===" << endl;
    testProducerConsumer();

    return 0;
}

```

20. 线程池的实现原理?

```

#include <iostream>
#include <vector>
#include <queue>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <functional>
#include <future>
using namespace std;

class ThreadPool {
    vector<thread> workers;
    queue<function<void()>> tasks;
    mutex mtx;
    condition_variable cv;
    bool stop;

public:
    ThreadPool(int numThreads) : stop(false) {
        for (int i = 0; i < numThreads; i++) {
            workers.emplace_back([this]() {
                while (true) {
                    function<void()> task;
                    {
                        unique_lock<mutex> lock(mtx);
                        cv.wait(lock, [this]() { return stop || !tasks.empty(); });

                        if (stop && tasks.empty()) return;
                        task = move(tasks.front());
                        tasks.pop();
                    }
                    task();
                }
            });
        }
    }

    template<typename F>
    future<void> enqueue(F &&f) {
        auto task = make_shared<packaged_task<void()>>(forward<F>(f));
        future<void> res = task->get_future();
        {
            unique_lock<mutex> lock(mtx);
            if (stop) throw runtime_error("线程池已停止");
            tasks.emplace([task]() { (*task)(); });
        }
    }
}

```

```

    }
    cv.notify_one();
    return res;
}

~ThreadPool() {
    {
        unique_lock<mutex> lock(mtx);
        stop = true;
    }
    cv.notify_all();
    for (auto &w : workers) w.join();
}

};

int main() {
    ThreadPool pool(4);
    vector<future<void>> results;

    for (int i = 0; i < 8; i++) {
        results.emplace_back(pool.enqueue([i]() {
            cout << "任务 " << i << " 在 thread "
                << this_thread::get_id() << " 执行" << endl;
            this_thread::sleep_for(chrono::milliseconds(100));
        }));
    }

    for (auto &r : results) r.get();

    return 0;
}

```

七、内存管理进阶

21. new/delete 和 malloc/free 的区别？

对比项	new/delete	malloc/free
本质	运算符 (C++)	函数 (C)
返回类型	具体类型指针	void*, 需强制转换
构造/析构	自动调用	不调用
失败处理	抛出 bad_alloc	返回 NULL
大小计算	自动计算	需要手动指定字节数
重载	可以重载	不能重载

```

#include <iostream>
#include <cstdlib>
using namespace std;

```

```

class MyClass {
public:
    MyClass() { cout << "构造" << endl; }
    ~MyClass() { cout << "析构" << endl; }
};

int main() {
    // new: 分配 + 构造
    MyClass *p1 = new MyClass();
    delete p1; // 析构 + 释放

    // malloc: 只分配, 不构造
    MyClass *p2 = (MyClass *)malloc(sizeof(MyClass));
    // 需要手动调用构造 (placement new)
    new(p2) MyClass();
    p2->~MyClass(); // 手动调用析构
    free(p2);       // 只释放, 不析构

    // new[] 和 delete[] 必须配对
    int *arr = new int[10];
    delete[] arr;

    return 0;
}

```

22. 内存泄漏如何检测和预防?

检测方法:

1. **Valgrind** (Linux) : `valgrind --leak-check=full ./program`
2. **AddressSanitizer** (GCC/Clang) : 编译加 `-fsanitize=address`
3. **Visual Studio 调试器**: `_CrtDumpMemoryLeaks()`
4. **智能指针**: 从根源避免

```

#include <iostream>
#include <memory>
using namespace std;

// 常见内存泄漏场景及修复
void leakExample1() {
    // 场景1: 忘记 delete
    int *p = new int(42);
    // 修复: 使用智能指针
    auto sp = make_unique<int>(42);
}

void leakExample2() {
    // 场景2: 异常导致跳过 delete
    int *p = new int(42);
    doSomething(); // 可能抛异常
    delete p;      // 异常时不会执行
    // 修复: RAII
    unique_ptr<int> sp(new int(42));
}

```

```

doSomething(); // 异常时自动释放
}

void leakExample3() {
    // 场景3: 重新赋值导致原内存丢失
    int *p = new int(1);
    p = new int(2); // 第一块内存泄漏
    // 修复: 先 delete 再赋值, 或使用智能指针
}

int main() {
    // 使用智能指针彻底避免内存泄漏
    unique_ptr<int> p1 = make_unique<int>(10);
    shared_ptr<int> p2 = make_shared<int>(20);

    cout << *p1 << ", " << *p2 << endl;
    return 0;
}

```

八、飞机大战项目技术点

23. 飞机大战项目中的 OOP 设计?

```

#include <iostream>
#include <vector>
#include <memory>
#include <string>
using namespace std;

// 游戏对象基类
class GameObject {
protected:
    int x, y;
    int width, height;
    bool alive;
public:
    GameObject(int x, int y, int w, int h)
        : x(x), y(y), width(w), height(h), alive(true) {}
    virtual ~GameObject() = default;
    virtual void update() = 0;
    virtual void render() = 0;
    bool isAlive() const { return alive; }
    // 碰撞检测
    bool collidesWith(const GameObject &other) const {
        return !(x + width < other.x || other.x + other.width < x ||
            y + height < other.y || other.y + other.height < y);
    }
};

// 玩家飞机
class Player : public GameObject {
    int hp;
    int score;
}

```

```

public:
    Player(int x, int y) : GameObject(x, y, 50, 50), hp(3), score(0) {}
    void update() override { /* 处理输入 */ }
    void render() override { cout << "渲染玩家" << endl; }
    void shoot(vector<shared_ptr<GameObject>> &bullets);
    int getScore() const { return score; }
};

// 敌机
class Enemy : public GameObject {
    int speed;
    int type;
public:
    Enemy(int x, int y, int type)
        : GameObject(x, y, 40, 40), speed(2), type(type) {}
    void update() override { y += speed; }
    void render() override { cout << "渲染敌机 type=" << type << endl; }
};

// 子弹
class Bullet : public GameObject {
    int speed;
    int damage;
public:
    Bullet(int x, int y) : GameObject(x, y, 10, 20), speed(8), damage(1) {}
    void update() override { y -= speed; }
    void render() override { cout << "渲染子弹" << endl; }
};

// 游戏管理器
class GameManager {
    shared_ptr<Player> player;
    vector<shared_ptr<GameObject>> enemies;
    vector<shared_ptr<GameObject>> bullets;
public:
    void start() {
        player = make_shared<Player>(200, 500);
        // 游戏主循环
        while (true) {
            update();
            render();
            checkCollisions();
        }
    }

    void update() {
        player->update();
        for (auto &e : enemies) e->update();
        for (auto &b : bullets) b->update();
        // 移除死亡对象
        enemies.erase(remove_if(enemies.begin(), enemies.end(),
            [](auto &e) { return !e->isAlive(); }), enemies.end());
    }

    void checkCollisions() {
        for (auto &b : bullets) {

```



```

        for (auto &e : enemies) {
            if (b->isAlive() && e->isAlive() && b->collideswith(*e)) {
                b->alive = false;
                e->alive = false;
                // 加分逻辑...
            }
        }
    }
}

void render() {
    player->render();
    for (auto &e : enemies) e->render();
    for (auto &b : bullets) b->render();
}
};

```

项目涉及知识点：

- **继承与多态**：GameObject 基类，Player/Enemy/Bullet 派生类
- **虚函数**：update()、render() 虚函数实现多态
- **智能指针**：shared_ptr 管理游戏对象生命周期
- **STL 容器**：vector 存储游戏对象
- **设计模式**：工厂模式创建敌机，观察者模式处理碰撞事件

九、IM 即时通讯项目技术点

24. IM 项目中的网络通信和多线程？

```

#include <iostream>
#include <string>
#include <vector>
#include <thread>
#include <mutex>
#include <queue>
#include <functional>
using namespace std;

// 消息结构
struct Message {
    string sender;
    string receiver;
    string content;
    int timestamp;
};

// 线程安全的消息队列
template<typename T>
class ThreadSafeQueue {
    queue<T> q;
    mutable mutex mtx;
    condition_variable cv;
};

```

```

public:
    void push(T item) {
        lock_guard<mutex> lock(mtx);
        q.push(move(item));
        cv.notify_one();
    }

    T pop() {
        unique_lock<mutex> lock(mtx);
        cv.wait(lock, [this]() { return !q.empty(); });
        T item = move(q.front());
        q.pop();
        return item;
    }

    bool tryPop(T &item) {
        lock_guard<mutex> lock(mtx);
        if (q.empty()) return false;
        item = move(q.front());
        q.pop();
        return true;
    }
};

// 网络客户端（简化版）
class IMClient {
    string username;
    ThreadSafeQueue<Message> sendQueue;
    ThreadSafeQueue<Message> recvQueue;
    thread sendThread;
    thread recvThread;
    bool running;

public:
    IMClient(string name) : username(name), running(false) {}

    void connect(const string &serverAddr, int port) {
        running = true;
        // 发送线程
        sendThread = thread([this]() {
            while (running) {
                Message msg = sendQueue.pop();
                // 发送到服务器...
                cout << "发送: " << msg.content << endl;
            }
        });

        // 接收线程
        recvThread = thread([this]() {
            while (running) {
                // 从服务器接收...
                Message msg;
                msg.sender = "server";
                msg.content = "欢迎 " + username;
                recvQueue.push(msg);
            }
        });
    }
};

```

```

    });
}

void sendMessage(const string &to, const string &content) {
    Message msg;
    msg.sender = username;
    msg.receiver = to;
    msg.content = content;
    sendQueue.push(msg);
}

void disconnect() {
    running = false;
    sendQueue.push(Message{}); // 唤醒阻塞的 pop
    recvQueue.push(Message{});
    if (sendThread.joinable()) sendThread.join();
    if (recvThread.joinable()) recvThread.join();
}
};

int main() {
    IMClient client("张三");
    client.connect("127.0.0.1", 8888);

    client.sendMessage("李四", "你好!");

    // 等待一段时间后断开
    this_thread::sleep_for(chrono::seconds(1));
    client.disconnect();

    return 0;
}

```

项目涉及知识点：

- **多线程**：发送线程、接收线程、UI 线程分离
- **线程同步**：mutex、condition_variable 保护共享数据
- **生产者消费者**：消息队列模型
- **网络编程**：Socket 通信（TCP）
- **Qt 信号与槽**：UI 更新、消息通知
- **设计模式**：单例（连接管理）、观察者（消息回调）

附录：C++ 学习路线图



常见面试题速查表

问题	核心答案
多态实现原理	虚函数表 (vtable) , 运行时通过 vptr 查找
智能指针种类	unique_ptr (独占)、shared_ptr (共享)、weak_ptr (观察)
vector 扩容	2 倍扩容, 拷贝/移动旧数据, 释放旧内存
map vs unordered_map	红黑树 $O(\log n)$ vs 哈希表 $O(1)$
深拷贝 vs 浅拷贝	独立内存 vs 共享指针
new vs malloc	运算符/调用构造 vs 函数/不构造
线程安全	mutex、lock_guard、atomic
单例模式	static local variable (C++11 线程安全)